

ML in Applications

***Dipartimento di Automatica e Informatica
Politecnico di Torino, Torino, ITALY***



**Politecnico
di Torino**

Lab 6

Uncertainty in DL

Uncertainty - recall

- **Epistemic** uncertainty refers to the uncertainty in **model parameters**. This type of uncertainty decreases when more data is given to the model
- **Aleatoric** uncertainty refers to **noise inherent in the observations**
 - Homoscedastic noise: noise is prevalent in the entire dataset
 - Heteroscedastic noise: noise is unique to each data-point
- In general, in big data regimes (e.g., deep learning with image data), it is most effective to model **aleatoric** uncertainty, which cannot be explained away [1]
- **Out-of-distribution** examples, which can be identified with epistemic uncertainty, cannot be identified with aleatoric uncertainty alone: **we need both**

[1] What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? (Kendall et al., 2017)

Uncertainty modelling

- Variational inference
- MC Dropout
- Ensemble
- Test-time data augmentation (TTA)

Epistemic

Aleatoric

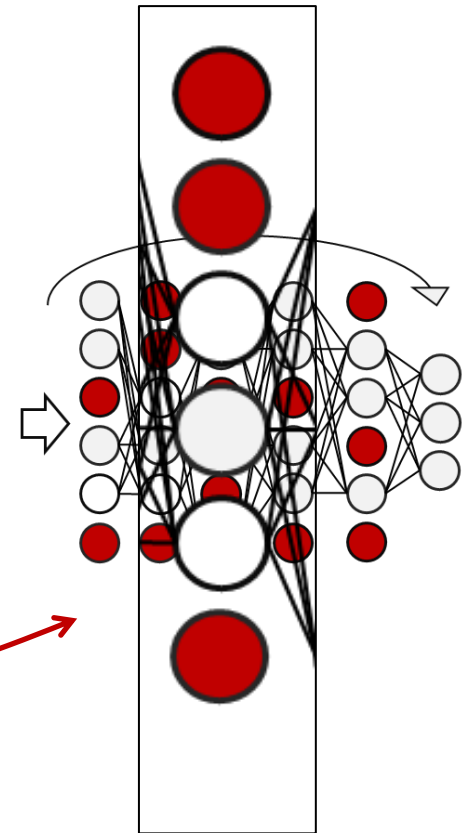
A note on terminology. The word epistemic comes from “episteme”, Greek for “knowledge”, i.e., epistemic uncertainty is “knowledge uncertainty”. Aleatoric comes from the Latin “*aleator*”, or “dice player”, i.e., aleatoric uncertainty is the “dice player’s” uncertainty [1].

[1] Y. Gal. Uncertainty in Deep Learning. PhD thesis, University of Cambridge, 2016

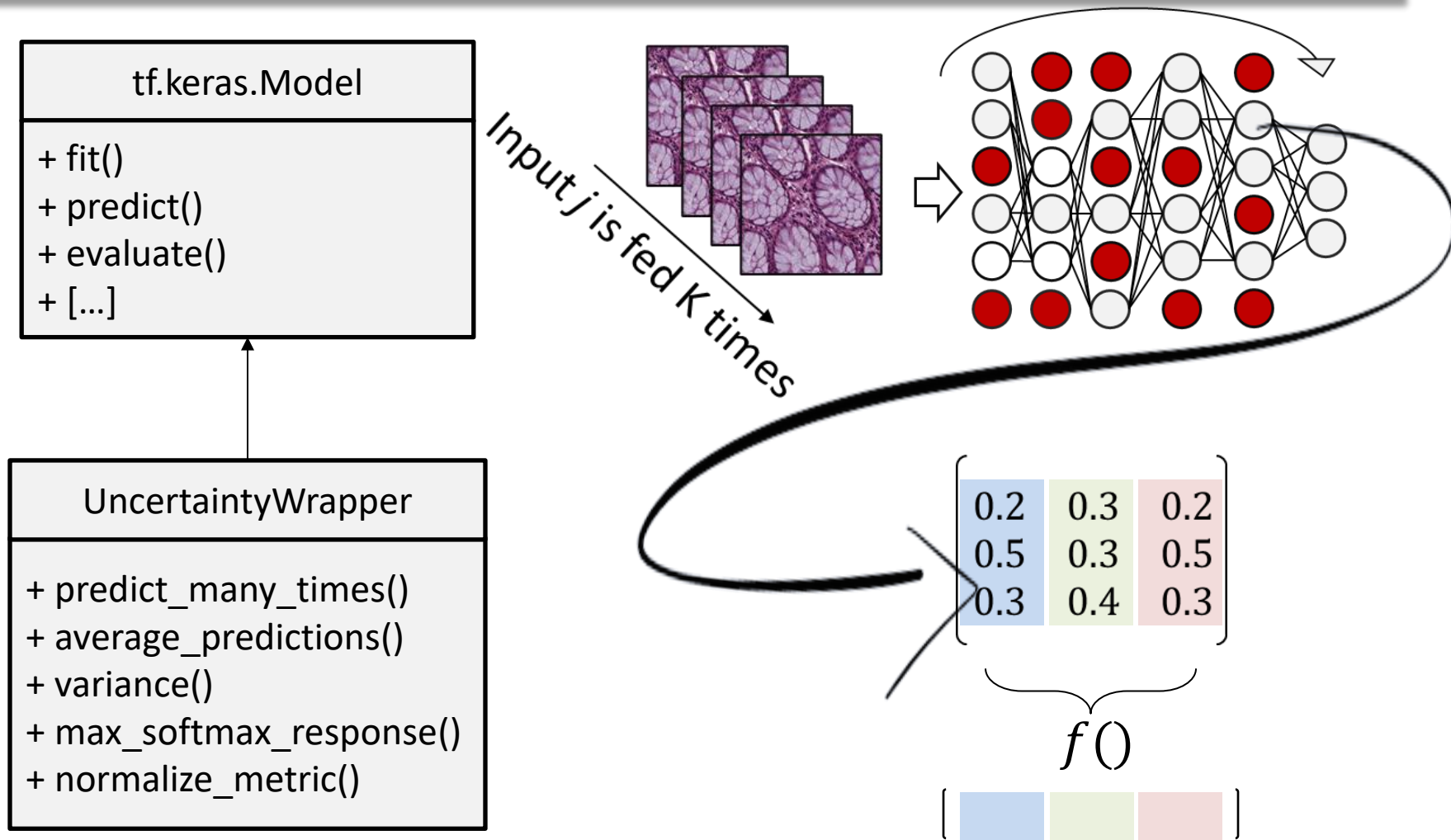
MC Dropout-based version of ResNet18

```
1 class ResNet(tf.keras.Model):
2
3     def __init__(self, num_classes=10):
4         super(ResNet, self).__init__()
5         self.block_a = tf.keras.Sequential(
6             [tf.keras.layers.Conv2D(64,
7                                     kernel_size=7,
8                                     strides=2,
9                                     padding='same'),
10              tf.keras.layers.BatchNormalization(),
11              tf.keras.layers.MaxPool2D(pool_size=3,
12                                       strides=2,
13                                       padding='same')])
14         self.block_b = ResnetBlock(64, 2, downscale=False)
15         self.block_c = ResnetBlock(128, 2)
16         self.block_d = ResnetBlock(256, 2)
17         self.block_e = ResnetBlock(512, 2)
18         self.global_pool = tf.keras.layers.GlobalAveragePooling2D()
19         self.classifier = tf.keras.layers.Dense(
20             num_classes,
21             activation="softmax")
22
23     def call(self, inputs):
24         x = self.block_a(inputs)
25         x = self.block_b(x)
26         x = self.block_c(x)
27         x = self.block_d(x)
28         x = self.block_e(x)
29         x = self.global_pool(x)
30         return self.classifier(x)
```

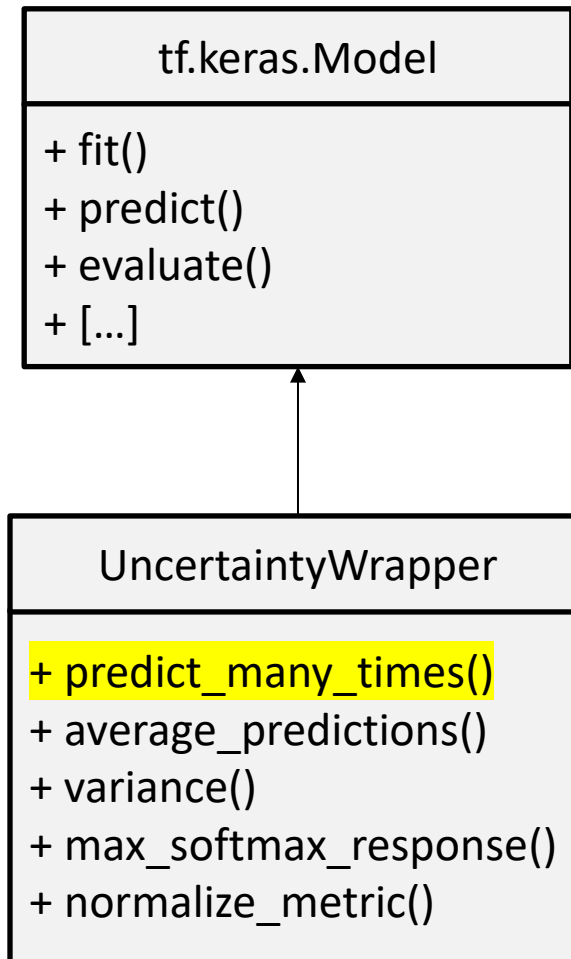
```
x = tf.keras.layers.Dropout(
    self.mc_dropout)(x, training=True)
```



Uncertainty wrapper for Keras models

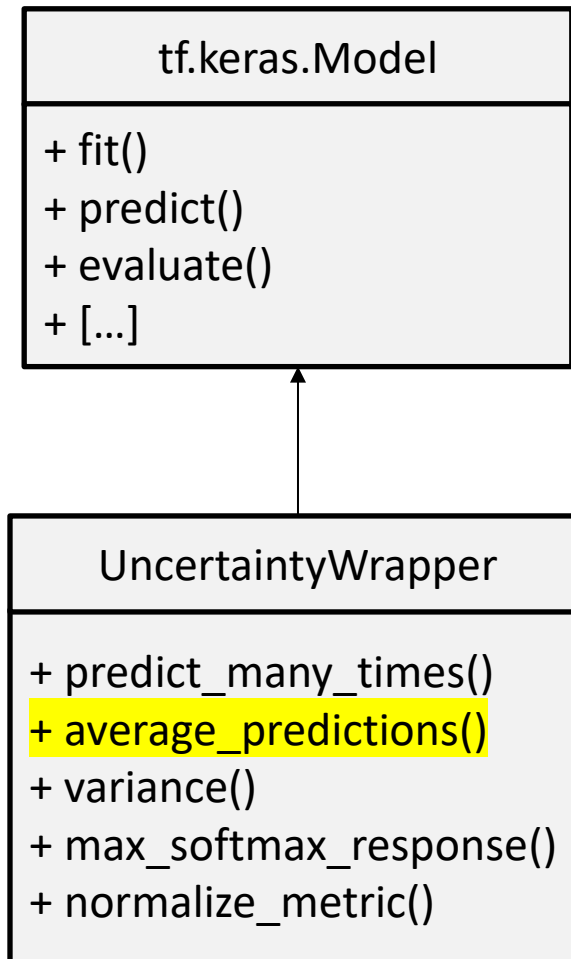


Uncertainty wrapper for Keras models



for i=1 to K do:
 model.predict

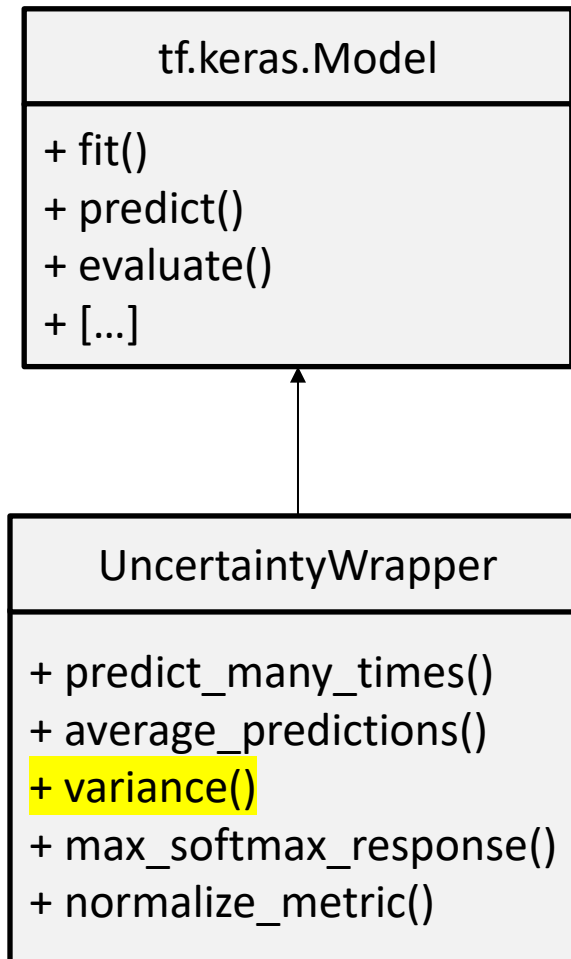
Uncertainty wrapper for Keras models



mean over draws

Data shape is (K, height, width, channels)

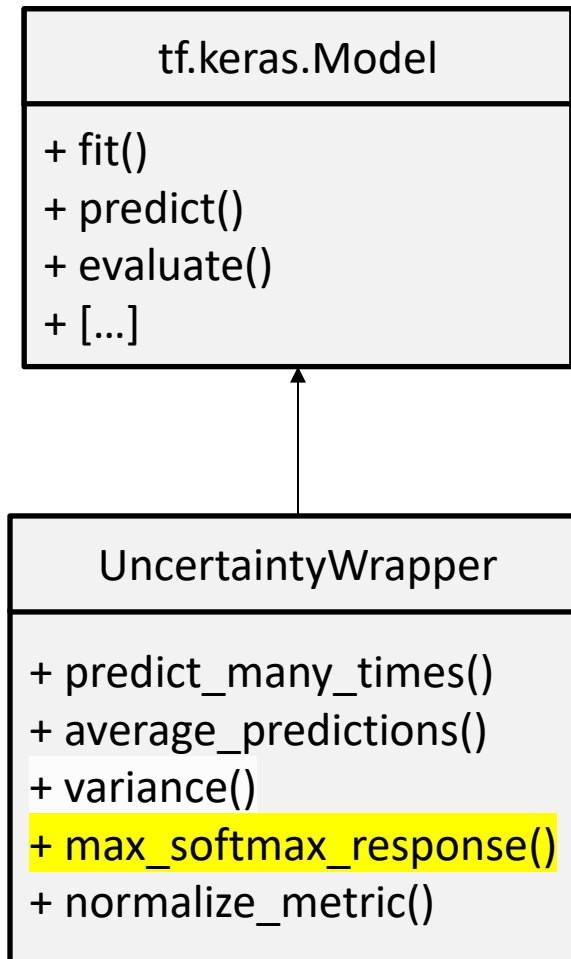
Uncertainty wrapper for Keras models



variance over draws

Data shape is (K, height, width, channels)

Uncertainty wrapper for Keras models



max of the **softmax** over draws
Data shape is (K, height, width, channels)

Uncertainty wrapper for TTA

